

# Machine Learning in Python

## Neural Networks II: Deep Learning, Transformers, and GenAI

Cristian A. Marocico, A. Emin Tatar

Center for Information Technology  
University of Groningen

Friday, February 20<sup>th</sup> 2026

# Outline

- 1 Convolutional Neural Networks (CNNs)
- 2 From RNNs to Transformers
- 3 Large Language Models (LLMs)
- 4 Generative AI and Modern Applications
- 5 Time-Series Forecasting with Feature-Based ML
- 6 Summary

# Why CNNs?

## The Problem with MLPs for Images

Definition

# Why CNNs?

## The Problem with MLPs for Images

Definition

Standard MLPs flatten images into vectors, causing two major problems:

- **Spatial Structure Lost:** A nose is defined by being *above* a mouth, not just by pixel values.

# Why CNNs?

## The Problem with MLPs for Images

Definition

Standard MLPs flatten images into vectors, causing two major problems:

- **Spatial Structure Lost:** A nose is defined by being *above* a mouth, not just by pixel values.
- **Parameter Explosion:** A  $1000 \times 1000$  image connected to 100 neurons requires **100 million weights**.

# The Convolution Operation

Filters and Feature Maps

Example

# The Convolution Operation

## Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be  $3 \times 3$  pixels.

# The Convolution Operation

## Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be  $3 \times 3$  pixels.
- It slides (convolves) over the image, computing dot products.



# The Convolution Operation

## Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be  $3 \times 3$  pixels.
- It slides (convolves) over the image, computing dot products.
- This creates a **Feature Map** showing where patterns appear.

# The Convolution Operation

## Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be  $3 \times 3$  pixels.
- It slides (convolves) over the image, computing dot products.
- This creates a **Feature Map** showing where patterns appear.
- Early filters detect edges; deeper filters detect complex shapes.

# Controlling Dimensions

## Padding and Stride

# Controlling Dimensions

## Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution:** No padding. Output shrinks ( $5 \times 5 \rightarrow 3 \times 3$ ).

# Controlling Dimensions

## Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution**: No padding. Output shrinks ( $5 \times 5 \rightarrow 3 \times 3$ ).
- **Padding**: Add zeros around border to maintain size.

# Controlling Dimensions

## Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution**: No padding. Output shrinks ( $5 \times 5 \rightarrow 3 \times 3$ ).
- **Padding**: Add zeros around border to maintain size.
- **Stride**: How many pixels to jump. Stride=2 usually reduces dimensions by about half.

# Controlling Dimensions

## Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution**: No padding. Output shrinks ( $5 \times 5 \rightarrow 3 \times 3$ ).
- **Padding**: Add zeros around border to maintain size.
- **Stride**: How many pixels to jump. Stride=2 usually reduces dimensions by about half.
- **Formula**:  $O = \frac{I-K+2P}{S} + 1$

# Pooling Layers

MaxPool2d

Example



# Pooling Layers

## MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.

# Pooling Layers

## MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.
- Asks: "Is there a feature in this patch? Yes/No."

# Pooling Layers

## MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.
- Asks: "Is there a feature in this patch? Yes/No."
- Provides translation invariance (feature can shift slightly).

# Pooling Layers

## MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.
- Asks: "Is there a feature in this patch? Yes/No."
- Provides translation invariance (feature can shift slightly).
- Typical:  $2 \times 2$  pool with stride 2 halves dimensions.

# Famous Architectures

## VGG and ResNet

# Famous Architectures

## VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small  $3 \times 3$  filters. Very deep.

# Famous Architectures

## VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small  $3 \times 3$  filters. Very deep.
- **ResNet (2015)**: Introduced **Skip Connections** (Residuals).

# Famous Architectures

## VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small  $3 \times 3$  filters. Very deep.
- **ResNet (2015)**: Introduced **Skip Connections** (Residuals).
- Skip connections let gradients "skip" layers to avoid vanishing.



# Famous Architectures

## VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small  $3 \times 3$  filters. Very deep.
- **ResNet (2015)**: Introduced **Skip Connections** (Residuals).
- Skip connections let gradients "skip" layers to avoid vanishing.
- Enables training networks with 100+ layers.

# Why This Matters

Sequence Data Is Everywhere

Definition

# Why This Matters

## Sequence Data Is Everywhere

Definition

Many research problems are sequential:

- Clinical notes and patient timelines

# Why This Matters

## Sequence Data Is Everywhere

Definition

Many research problems are sequential:

- Clinical notes and patient timelines
- Instrument logs and time-ordered measurements

# Why This Matters

## Sequence Data Is Everywhere

Definition

Many research problems are sequential:

- Clinical notes and patient timelines
- Instrument logs and time-ordered measurements
- Reports, interviews, and long documents

# Why This Matters

## Sequence Data Is Everywhere

Definition

Many research problems are sequential:

- Clinical notes and patient timelines
- Instrument logs and time-ordered measurements
- Reports, interviews, and long documents
- Key question: *which earlier tokens matter right now?*

## Running Example

| Clinical Sentence | Example |
|-------------------|---------|
|                   |         |

## Running Example

Clinical Sentence

Example

*"Patient admitted three days ago with acute chest pain, treated with aspirin, now reports reduced discomfort."*



## Running Example

### Clinical Sentence

### Example

*"Patient admitted three days ago with acute chest pain, treated with aspirin, now reports reduced discomfort."* When interpreting "**discomfort**", the model should retrieve context from:

- "chest pain"

## Running Example

### Clinical Sentence

### Example

*"Patient admitted three days ago with acute chest pain, treated with aspirin, now reports reduced discomfort."* When interpreting "**discomfort**", the model should retrieve context from:

- "chest pain"
- "aspirin"

# Running Example

## Clinical Sentence

## Example

*"Patient admitted three days ago with acute chest pain, treated with aspirin, now reports reduced discomfort."* When interpreting "**discomfort**", the model should retrieve context from:

- "chest pain"
- "aspirin"
- other relevant earlier tokens

# RNNs: First Main Solution

## Sequential Hidden-State Memory

# RNNs: First Main Solution

## Sequential Hidden-State Memory

RNN update:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

- $x_t$ : current token representation

# RNNs: First Main Solution

## Sequential Hidden-State Memory

RNN update:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

- $x_t$ : current token representation
- $h_{t-1}$ : previous memory

# RNNs: First Main Solution

## Sequential Hidden-State Memory

RNN update:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

- $x_t$ : current token representation
- $h_{t-1}$ : previous memory
- $h_t$ : updated memory

# RNNs: First Main Solution

## Sequential Hidden-State Memory

RNN update:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

- $x_t$ : current token representation
- $h_{t-1}$ : previous memory
- $h_t$ : updated memory
- Strength: natural for ordered data



# RNNs: First Main Solution

## Sequential Hidden-State Memory

RNN update:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

- $x_t$ : current token representation
- $h_{t-1}$ : previous memory
- $h_t$ : updated memory
- Strength: natural for ordered data
- Limits: sequential bottleneck + fading long-range memory

# RNNs: First Main Solution

## Sequential Hidden-State Memory

RNN update:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

- $x_t$ : current token representation
- $h_{t-1}$ : previous memory
- $h_t$ : updated memory
- Strength: natural for ordered data
- Limits: sequential bottleneck + fading long-range memory
- LSTMs improved memory with gates, but remained sequential

# Attention as Weighted Lookup

Query-Specific Relevance Weights

Example

# Attention as Weighted Lookup

## Query-Specific Relevance Weights

Example

Attention replaces "single compressed memory" with dynamic lookup.

- Weights are non-negative and sum to 1

# Attention as Weighted Lookup

## Query-Specific Relevance Weights

Example

Attention replaces "single compressed memory" with dynamic lookup.

- Weights are non-negative and sum to 1
- Different query tokens produce different weight patterns

# Attention as Weighted Lookup

## Query-Specific Relevance Weights

Example

Attention replaces "single compressed memory" with dynamic lookup.

- Weights are non-negative and sum to 1
- Different query tokens produce different weight patterns
- For query token "today", weights are distributed across ["Patient", "reports", "chest", "pain", "today"]

# Attention as Weighted Lookup

## Query-Specific Relevance Weights

Example

Attention replaces "single compressed memory" with dynamic lookup.

- Weights are non-negative and sum to 1
- Different query tokens produce different weight patterns
- For query token "today", weights are distributed across ["Patient", "reports", "chest", "pain", "today"]
- Self-weight can be  $\leq 1$  because context tokens also contribute

# Transformers: Attention-Centered Architecture

"Attention Is All You Need"

Definition



# Transformers: Attention-Centered Architecture

## "Attention Is All You Need"

Definition

Transformers remove recurrence and rely on attention:

| Feature       | RNN / LSTM                     | Transformer                                  |
|---------------|--------------------------------|----------------------------------------------|
| Processing    | Sequential                     | Parallel in training                         |
| Context       | Compressed in one hidden state | Direct weighted access across positions      |
| Long distance | Degrades with sequence length  | One attention step away (subject to masking) |

# Core Formula and Transformer Block

Q, K, V and Scaled Dot Product

# Core Formula and Transformer Block

## Q, K, V and Scaled Dot Product

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Query: what this position is looking for

# Core Formula and Transformer Block

## Q, K, V and Scaled Dot Product

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Query: what this position is looking for
- Key: what each position offers

# Core Formula and Transformer Block

## Q, K, V and Scaled Dot Product

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Query: what this position is looking for
- Key: what each position offers
- Value: information to aggregate

# Core Formula and Transformer Block

## Q, K, V and Scaled Dot Product

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

- Query: what this position is looking for
- Key: what each position offers
- Value: information to aggregate
- $\sqrt{d_k}$  scaling stabilizes softmax/gradients

# Core Formula and Transformer Block

## Q, K, V and Scaled Dot Product

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

- Query: what this position is looking for
- Key: what each position offers
- Value: information to aggregate
- $\sqrt{d_k}$  scaling stabilizes softmax/gradients
- Block components: self-attention, feed-forward, residual, layer norm, positional encoding

# Multi-Head and Model Families

Parallel Attention Heads

Example



# Multi-Head and Model Families

## Parallel Attention Heads

Example

Multi-head attention runs several Q/K/V projections in parallel.

- Different heads can focus on different relations (syntax, semantics, locality)

# Multi-Head and Model Families

## Parallel Attention Heads

Example

Multi-head attention runs several Q/K/V projections in parallel.

- Different heads can focus on different relations (syntax, semantics, locality)
- Outputs are concatenated and projected back to model dimension

# Multi-Head and Model Families

## Parallel Attention Heads

Example

Multi-head attention runs several Q/K/V projections in parallel.

- Different heads can focus on different relations (syntax, semantics, locality)
- Outputs are concatenated and projected back to model dimension
- Encoder-only: BERT (understanding/classification)

# Multi-Head and Model Families

## Parallel Attention Heads

Example

Multi-head attention runs several Q/K/V projections in parallel.

- Different heads can focus on different relations (syntax, semantics, locality)
- Outputs are concatenated and projected back to model dimension
- Encoder-only: BERT (understanding/classification)
- Decoder-only: GPT (generation with causal masking)

# Multi-Head and Model Families

## Parallel Attention Heads

Example

Multi-head attention runs several Q/K/V projections in parallel.

- Different heads can focus on different relations (syntax, semantics, locality)
- Outputs are concatenated and projected back to model dimension
- Encoder-only: BERT (understanding/classification)
- Decoder-only: GPT (generation with causal masking)
- Encoder-decoder: T5 (translation/summarization)

# Why GPT Is Left-to-Right

## Causal Masking

# Why GPT Is Left-to-Right

## Causal Masking

For token  $t$ , attention is allowed only to tokens  $\leq t$ .

- Future positions are masked out

# Why GPT Is Left-to-Right

## Causal Masking

For token  $t$ , attention is allowed only to tokens  $\leq t$ .

- Future positions are masked out
- Training remains parallel across positions (with masking)



# Why GPT Is Left-to-Right

## Causal Masking

For token  $t$ , attention is allowed only to tokens  $\leq t$ .

- Future positions are masked out
- Training remains parallel across positions (with masking)
- Generation remains token-by-token

# Why GPT Is Left-to-Right

## Causal Masking

For token  $t$ , attention is allowed only to tokens  $\leq t$ .

- Future positions are masked out
- Training remains parallel across positions (with masking)
- Generation remains token-by-token
- Early tokens have less context than later tokens

# Practical Research Workflow

Pretrained-First Strategy

Example

# Practical Research Workflow

## Pretrained-First Strategy

Example

Most researchers do not train transformers from scratch:

- Choose a pretrained model for the task/domain

# Practical Research Workflow

## Pretrained-First Strategy

Example

Most researchers do not train transformers from scratch:

- Choose a pretrained model for the task/domain
- Start with prompting and retrieval

# Practical Research Workflow

## Pretrained-First Strategy

Example

Most researchers do not train transformers from scratch:

- Choose a pretrained model for the task/domain
- Start with prompting and retrieval
- Evaluate on domain-specific examples

# Practical Research Workflow

## Pretrained-First Strategy

Example

Most researchers do not train transformers from scratch:

- Choose a pretrained model for the task/domain
- Start with prompting and retrieval
- Evaluate on domain-specific examples
- Fine-tune only if needed

# Practical Research Workflow

## Pretrained-First Strategy

Example

Most researchers do not train transformers from scratch:

- Choose a pretrained model for the task/domain
- Start with prompting and retrieval
- Evaluate on domain-specific examples
- Fine-tune only if needed
- Keep expert review in high-stakes settings



# Evolution of LLMs

Scale as the Main Driver

Definition

# Evolution of LLMs

## Scale as the Main Driver

Definition

LLM progress accelerated with scale (parameters, data, compute):

| Model   | Year | Parameters  | Training Data | Notable Point                      |
|---------|------|-------------|---------------|------------------------------------|
| BERT    | 2018 | 340M        | 16GB          | Strong "understanding" performance |
| GPT-2   | 2019 | 1.5B        | 40GB          | High-quality open-ended generation |
| GPT-3   | 2020 | 175B        | 570GB         | Emergent few-shot behavior         |
| Llama-2 | 2023 | 70B         | 2TB           | Open-weight ecosystem growth       |
| GPT-4   | 2023 | Undisclosed | Unknown       | Strong multimodal capabilities     |

# Architecture Families

Encoder, Decoder, Encoder-Decoder

# Architecture Families

## Encoder, Decoder, Encoder-Decoder

- BERT (Encoder-only): understanding tasks (classification, QA)

# Architecture Families

## Encoder, Decoder, Encoder-Decoder

- BERT (Encoder-only): understanding tasks (classification, QA)
- GPT (Decoder-only): generation tasks (next-token prediction)

# Architecture Families

## Encoder, Decoder, Encoder-Decoder

- BERT (Encoder-only): understanding tasks (classification, QA)
- GPT (Decoder-only): generation tasks (next-token prediction)
- T5 (Encoder-Decoder): text-to-text tasks (translation, summarization)

# Architecture Families

## Encoder, Decoder, Encoder-Decoder

- **BERT (Encoder-only)**: understanding tasks (classification, QA)
- **GPT (Decoder-only)**: generation tasks (next-token prediction)
- **T5 (Encoder-Decoder)**: text-to-text tasks (translation, summarization)
- **Scaling hypothesis**: performance improves predictably with more parameters, data, and compute

# The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example



# The LLM Lifecycle

## Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.

# The LLM Lifecycle

## Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.

# The LLM Lifecycle

## Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.
  - **SFT**: Train on Q&A pairs to make it a chatbot.

# The LLM Lifecycle

## Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.
  - **SFT**: Train on Q&A pairs to make it a chatbot.
  - **RLHF**: Align to be helpful and harmless.

# The LLM Lifecycle

## Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.
  - **SFT**: Train on Q&A pairs to make it a chatbot.
  - **RLHF**: Align to be helpful and harmless.
- **Inference**: Using the model. Feed prompt, generate response.

# Tokenization and Context Windows

## BPE and Subword Units

Example

# Tokenization and Context Windows

## BPE and Subword Units

Example

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.

# Tokenization and Context Windows

## BPE and Subword Units

Example

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.
- **Rough rule**: 1000 tokens  $\approx$  750 words.



# Tokenization and Context Windows

## BPE and Subword Units

Example

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.
- **Rough rule**: 1000 tokens  $\approx$  750 words.
- BPE/subword tokenization handles rare or unseen words by splitting them.

# Tokenization and Context Windows

## BPE and Subword Units

Example

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.
- **Rough rule**: 1000 tokens  $\approx$  750 words.
- BPE/subword tokenization handles rare or unseen words by splitting them.
- **Context Window**: Limit on how much text the model can use at once (model/version dependent).

# Next-Token Prediction Mechanics

## From Logits to Text

# Next-Token Prediction Mechanics

## From Logits to Text

At each decoding step, the model outputs logits  $z \in \mathbb{R}^{|V|}$  over the vocabulary.

# Next-Token Prediction Mechanics

## From Logits to Text

At each decoding step, the model outputs logits  $z \in \mathbb{R}^{|V|}$  over the vocabulary. Probabilities are computed as:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

# Next-Token Prediction Mechanics

## From Logits to Text

At each decoding step, the model outputs logits  $z \in \mathbb{R}^{|V|}$  over the vocabulary. Probabilities are computed as:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- This is a full distribution over all candidate next tokens.

# Next-Token Prediction Mechanics

## From Logits to Text

At each decoding step, the model outputs logits  $z \in \mathbb{R}^{|V|}$  over the vocabulary. Probabilities are computed as:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- This is a full distribution over all candidate next tokens.
- Decoding selects/samples one token from this distribution.

# Next-Token Prediction Mechanics

## From Logits to Text

At each decoding step, the model outputs logits  $z \in \mathbb{R}^{|V|}$  over the vocabulary. Probabilities are computed as:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- This is a full distribution over all candidate next tokens.
- Decoding selects/samples one token from this distribution.
- The selected token is appended, and the process repeats autoregressively.



# Sampling Controls

Temperature, Top-k, Top-p

Example

# Sampling Controls

## Temperature, Top-k, Top-p

Example

Generation quality/style depends on sampling strategy:

| Parameter   | Effect                                                  | Typical |
|-------------|---------------------------------------------------------|---------|
| Temperature | Flattens/sharpens token probabilities                   | 0.7     |
| Top-k       | Restricts choices to k most likely tokens               | 50      |
| Top-p       | Restricts to smallest set with cumulative prob $\geq p$ | 0.9     |

# Sampling Controls

## Temperature, Top-k, Top-p

Example

Generation quality/style depends on sampling strategy:

| Parameter   | Effect                                                  | Typical |
|-------------|---------------------------------------------------------|---------|
| Temperature | Flattens/sharpens token probabilities                   | 0.7     |
| Top-k       | Restricts choices to k most likely tokens               | 50      |
| Top-p       | Restricts to smallest set with cumulative prob $\geq p$ | 0.9     |

- Low temperature: focused/deterministic

# Sampling Controls

## Temperature, Top-k, Top-p

Example

Generation quality/style depends on sampling strategy:

| Parameter   | Effect                                                  | Typical |
|-------------|---------------------------------------------------------|---------|
| Temperature | Flattens/sharpens token probabilities                   | 0.7     |
| Top-k       | Restricts choices to k most likely tokens               | 50      |
| Top-p       | Restricts to smallest set with cumulative prob $\geq p$ | 0.9     |

- Low temperature: focused/deterministic
- High temperature: diverse/chaotic

# Sampling Controls

## Temperature, Top-k, Top-p

Example

Generation quality/style depends on sampling strategy:

| Parameter   | Effect                                                  | Typical |
|-------------|---------------------------------------------------------|---------|
| Temperature | Flattens/sharpens token probabilities                   | 0.7     |
| Top-k       | Restricts choices to k most likely tokens               | 50      |
| Top-p       | Restricts to smallest set with cumulative prob $\geq p$ | 0.9     |

- Low temperature: focused/deterministic
- High temperature: diverse/chaotic
- Top-k/top-p reduce low-probability tail sampling

# Prompt Engineering

## Guiding the Predictor

Example

# Prompt Engineering

## Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

# Prompt Engineering

## Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

- **Zero-Shot:** Just asking. "Translate to Spanish: Hello"



# Prompt Engineering

## Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

- **Zero-Shot**: Just asking. "Translate to Spanish: Hello"
- **Few-Shot**: Give examples to induce pattern-following behavior.

# Prompt Engineering

## Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

- **Zero-Shot**: Just asking. "Translate to Spanish: Hello"
- **Few-Shot**: Give examples to induce pattern-following behavior.
- **Chain of Thought**: "Let's think step by step." Can help some multi-step tasks (model/task dependent).

# Text Generation

It's Just Statistics

Definition

# Text Generation

## It's Just Statistics

### Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**:

# Text Generation

## It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%

# Text Generation

## It's Just Statistics

### Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%
- "gray": 15%

# Text Generation

## It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%
- "gray": 15%
- "falling": 1%

# Text Generation

## It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%
- "gray": 15%
- "falling": 1%
- It samples from this distribution to generate text.



# Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

# Retrieval-Augmented Generation (RAG)

## Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.

# Retrieval-Augmented Generation (RAG)

## Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.

# Retrieval-Augmented Generation (RAG)

## Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- One practical mitigation is **RAG**, which connects LLMs to external knowledge:

# Retrieval-Augmented Generation (RAG)

## Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- One practical mitigation is **RAG**, which connects LLMs to external knowledge:
  - ① **Retrieve**: Search database for relevant documents.

# Retrieval-Augmented Generation (RAG)

## Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- One practical mitigation is **RAG**, which connects LLMs to external knowledge:
  - 1 **Retrieve**: Search database for relevant documents.
  - 2 **Augment**: Paste documents into prompt context.

# Retrieval-Augmented Generation (RAG)

## Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- One practical mitigation is **RAG**, which connects LLMs to external knowledge:
  - 1 **Retrieve**: Search database for relevant documents.
  - 2 **Augment**: Paste documents into prompt context.
  - 3 **Generate**: LLM answers using provided context.

# Diffusion Models

## Image Generation (Stable Diffusion, DALL-E)



# Diffusion Models

## Image Generation (Stable Diffusion, DALL-E)

Image generation works differently from LLMs, using **Diffusion**:

- **Forward Process (Training)**: Take clear image, slowly add noise until pure static. Model learns destruction pattern.

# Diffusion Models

## Image Generation (Stable Diffusion, DALL-E)

Image generation works differently from LLMs, using **Diffusion**:

- **Forward Process (Training)**: Take clear image, slowly add noise until pure static. Model learns destruction pattern.
- **Reverse Process (Generation)**: Start with noise. Ask: "If this was a cat, how would you remove noise?" Repeat 50 times.

# Diffusion Models

## Image Generation (Stable Diffusion, DALL-E)

Image generation works differently from LLMs, using **Diffusion**:

- **Forward Process (Training)**: Take clear image, slowly add noise until pure static. Model learns destruction pattern.
- **Reverse Process (Generation)**: Start with noise. Ask: "If this was a cat, how would you remove noise?" Repeat 50 times.
- **Result**: Clear image emerges from noise.

# Ethical Considerations

Bias, Hallucination, and Copyright

Example

# Ethical Considerations

## Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.

# Ethical Considerations

## Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.
- **Hallucination**: LLMs optimize for plausibility, not truth. Dangerous for medical/legal advice.

# Ethical Considerations

## Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.
- **Hallucination**: LLMs optimize for plausibility, not truth. Dangerous for medical/legal advice.
- **Copyright**: Image models trained on artists' work. Code models on GitHub. Legal framework still evolving.

# Ethical Considerations

## Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.
- **Hallucination**: LLMs optimize for plausibility, not truth. Dangerous for medical/legal advice.
- **Copyright**: Image models trained on artists' work. Code models on GitHub. Legal framework still evolving.
- **Mitigation**: Data curation, RLHF, human oversight.



# Forecasting Problem Setup

| Formal Objective | Definition |
|------------------|------------|
|                  |            |

# Forecasting Problem Setup

## Formal Objective

Definition

For a univariate series  $\{y_t\}_{t=1}^T$ , one-step forecasting is:

$$\hat{y}_{t+1|t} = f_{\theta}(\phi_t)$$

where  $\phi_t$  uses only information available at time  $t$ .

# Forecasting Problem Setup

## Formal Objective

Definition

For a univariate series  $\{y_t\}_{t=1}^T$ , one-step forecasting is:

$$\hat{y}_{t+1|t} = f_{\theta}(\phi_t)$$

where  $\phi_t$  uses only information available at time  $t$ . Training is empirical risk minimization:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_{t \in \mathcal{T}_{train}} \mathcal{L}(y_{t+1}, f_{\theta}(\phi_t))$$

# Why Time Series Is Not i.i.d.

## Temporal Dependence and Structure

# Why Time Series Is Not i.i.d.

## Temporal Dependence and Structure

Time series typically combines:

$$y_t = T_t + S_t + R_t$$

# Why Time Series Is Not i.i.d.

## Temporal Dependence and Structure

Time series typically combines:

$$y_t = T_t + S_t + R_t$$

- $T_t$ : trend (long-term movement)

# Why Time Series Is Not i.i.d.

## Temporal Dependence and Structure

Time series typically combines:

$$y_t = T_t + S_t + R_t$$

- $T_t$ : trend (long-term movement)
- $S_t$ : seasonality (repeating calendar pattern)

# Why Time Series Is Not i.i.d.

## Temporal Dependence and Structure

Time series typically combines:

$$y_t = T_t + S_t + R_t$$

- $T_t$ : trend (long-term movement)
- $S_t$ : seasonality (repeating calendar pattern)
- $R_t$ : residual/noise



# Why Random Splits Fail

Leakage Through Time Mixing

Example

# Why Random Splits Fail

## Leakage Through Time Mixing

Example

Information sets are ordered:

$$\mathcal{F}_{t_1} \subset \mathcal{F}_{t_2}, \quad t_1 < t_2$$

# Why Random Splits Fail

## Leakage Through Time Mixing

Example

Information sets are ordered:

$$\mathcal{F}_{t_1} \subset \mathcal{F}_{t_2}, \quad t_1 < t_2$$

Random shuffling violates this causal flow and can leak future-conditioned patterns into training.

# Why Random Splits Fail

## Leakage Through Time Mixing

Example

Information sets are ordered:

$$\mathcal{F}_{t_1} \subset \mathcal{F}_{t_2}, \quad t_1 < t_2$$

Random shuffling violates this causal flow and can leak future-conditioned patterns into training.

- Correct: train on past, validate on future block.

# Why Random Splits Fail

## Leakage Through Time Mixing

Example

Information sets are ordered:

$$\mathcal{F}_{t_1} \subset \mathcal{F}_{t_2}, \quad t_1 < t_2$$

Random shuffling violates this causal flow and can leak future-conditioned patterns into training.

- Correct: train on past, validate on future block.
- Incorrect: random train/test split across timestamps.

# Strong Baselines First

Naive and Seasonal Naive

Definition

# Strong Baselines First

## Naive and Seasonal Naive

Definition

$$\hat{y}_{t+1}^{naive} = y_t, \quad \hat{y}_{t+1}^{seasonal} = y_{t+1-s}$$

# Strong Baselines First

## Naive and Seasonal Naive

Definition

$$\hat{y}_{t+1}^{naive} = y_t, \quad \hat{y}_{t+1}^{seasonal} = y_{t+1-s}$$

for monthly data,  $s = 12$ .



# Strong Baselines First

## Naive and Seasonal Naive

Definition

$$\hat{y}_{t+1}^{naive} = y_t, \quad \hat{y}_{t+1}^{seasonal} = y_{t+1-s}$$

for monthly data,  $s = 12$ .

- Random walk  $y_t = y_{t-1} + \varepsilon_t \rightarrow$  naive is optimal (MSE).

# Strong Baselines First

## Naive and Seasonal Naive

Definition

$$\hat{y}_{t+1}^{naive} = y_t, \quad \hat{y}_{t+1}^{seasonal} = y_{t+1-s}$$

for monthly data,  $s = 12$ .

- Random walk  $y_t = y_{t-1} + \varepsilon_t \rightarrow$  naive is optimal (MSE).
- Seasonal random walk  $y_t = y_{t-s} + \varepsilon_t \rightarrow$  seasonal naive is optimal.

# Strong Baselines First

## Naive and Seasonal Naive

Definition

$$\hat{y}_{t+1}^{naive} = y_t, \quad \hat{y}_{t+1}^{seasonal} = y_{t+1-s}$$

for monthly data,  $s = 12$ .

- Random walk  $y_t = y_{t-1} + \varepsilon_t \rightarrow$  naive is optimal (MSE).
- Seasonal random walk  $y_t = y_{t-s} + \varepsilon_t \rightarrow$  seasonal naive is optimal.
- If ML cannot beat these, complexity is not justified.

# Feature Engineering with Temporal Constraints

Building  $\phi_t$  Safely

# Feature Engineering with Temporal Constraints

## Building $\phi_t$ Safely

Example feature vector:

$$\phi_t = [y_{t-1}, y_{t-2}, y_{t-12}, \mu_{t-1}^{(3)}, \mu_{t-1}^{(12)}, \sin(2\pi m_t/12), \cos(2\pi m_t/12), t]$$

with rolling mean

$$\mu_{t-1}^{(w)} = \frac{1}{w} \sum_{j=1}^w y_{t-j}$$

- Shift first, then roll (prevents target leakage).

# Feature Engineering with Temporal Constraints

## Building $\phi_t$ Safely

Example feature vector:

$$\phi_t = [y_{t-1}, y_{t-2}, y_{t-12}, \mu_{t-1}^{(3)}, \mu_{t-1}^{(12)}, \sin(2\pi m_t/12), \cos(2\pi m_t/12), t]$$

with rolling mean

$$\mu_{t-1}^{(w)} = \frac{1}{w} \sum_{j=1}^w y_{t-j}$$

- Shift first, then roll (prevents target leakage).
- Cyclical encoding respects December-January adjacency.

# Model Families and Objectives

Ridge vs Boosting

Example

# Model Families and Objectives

## Ridge vs Boosting

Example

- Ridge: stable and interpretable with correlated lags. Ridge objective:

$$\hat{\beta} = \arg \min_{\beta} \left[ \frac{1}{n} |y - X\beta|^2 + \alpha |\beta|^2 \right]$$

- Boosting: captures nonlinear interactions. Boosting (additive model):

$$f_M(x) = \sum_{m=1}^M \eta h_m(x), \quad r_i^{(m)} \approx y_i - f_{m-1}(x_i)$$



# Walk-Forward Backtesting

Estimating Future Performance

Example

# Walk-Forward Backtesting

## Estimating Future Performance

Example

Choose ordered cutoffs  $\tau_1 < \dots < \tau_K$ . For fold  $k$ :

- Train on  $\{1, \dots, \tau_k\}$

# Walk-Forward Backtesting

## Estimating Future Performance

Example

Choose ordered cutoffs  $\tau_1 < \dots < \tau_K$ . For fold  $k$ :

- Train on  $\{1, \dots, \tau_k\}$
- Test on  $\{\tau_k + 1, \dots, \tau_k + h\}$

# Walk-Forward Backtesting

## Estimating Future Performance

Example

Choose ordered cutoffs  $\tau_1 < \dots < \tau_K$ . For fold  $k$ :

- Train on  $\{1, \dots, \tau_k\}$
- Test on  $\{\tau_k + 1, \dots, \tau_k + h\}$
- Compute metric  $M_k$

# Walk-Forward Backtesting

## Estimating Future Performance

Example

Choose ordered cutoffs  $\tau_1 < \dots < \tau_K$ . For fold  $k$ :

- Train on  $\{1, \dots, \tau_k\}$
- Test on  $\{\tau_k + 1, \dots, \tau_k + h\}$
- Compute metric  $M_k$

# Walk-Forward Backtesting

## Estimating Future Performance

Example

Choose ordered cutoffs  $\tau_1 < \dots < \tau_K$ . For fold  $k$ :

- Train on  $\{1, \dots, \tau_k\}$
- Test on  $\{\tau_k + 1, \dots, \tau_k + h\}$
- Compute metric  $M_k$

Aggregate:

$$\bar{M} = \frac{1}{K} \sum_{k=1}^K M_k$$

This is usually more reliable than a single holdout split.

# Diagnostics After Forecasting

## Residual Checks

# Diagnostics After Forecasting

## Residual Checks

Even with good MAE/RMSE, verify residual structure.

- Residual mean near zero (bias check).



# Diagnostics After Forecasting

## Residual Checks

Even with good MAE/RMSE, verify residual structure.

- Residual mean near zero (bias check).
- No strong temporal pattern in residual plots.

# Diagnostics After Forecasting

## Residual Checks

Even with good MAE/RMSE, verify residual structure.

- Residual mean near zero (bias check).
- No strong temporal pattern in residual plots.
- Low residual autocorrelation at small lags.

# Diagnostics After Forecasting

## Residual Checks

Even with good MAE/RMSE, verify residual structure.

- Residual mean near zero (bias check).
- No strong temporal pattern in residual plots.
- Low residual autocorrelation at small lags.
- If autocorrelation remains, predictive signal is still unmodeled.

# Key Takeaways

## Neural Networks II

# Key Takeaways

## Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.

# Key Takeaways

## Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.

# Key Takeaways

## Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.

# Key Takeaways

## Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.
- 4 **Time-Series ML**: Formalize  $f_{\theta}(\phi_t)$ , beat strong baselines, and validate with walk-forward backtesting.



# Key Takeaways

## Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.
- 4 **Time-Series ML**: Formalize  $f_{\theta}(\phi_t)$ , beat strong baselines, and validate with walk-forward backtesting.
- 5 **Generative AI**: Text (next-token), Images (diffusion). RAG grounds LLMs in facts.

# Key Takeaways

## Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.
- 4 **Time-Series ML**: Formalize  $f_{\theta}(\phi_t)$ , beat strong baselines, and validate with walk-forward backtesting.
- 5 **Generative AI**: Text (next-token), Images (diffusion). RAG grounds LLMs in facts.
- 6 **Ethics**: AI reflects training data. Requires human oversight and responsibility.